CLC TP3 Number 20250328

UDC 004 Available for reference □Yes □No

Southern University of Science and Technology

Undergraduate thesis report

Thesis Title: **ABIscope —Differential Analysis of Linux ABI Compatibility in Rust-based Kernels**

Student Name: **杨若谷**

Student ID: **12213043**

Department: **Computer Science and Engineering**

Program: **Computer Science and Technology**

Thesis Advisor: **张殷乾**

Apr 29, 2026

COMMITMENT OF HONESTY

1. I solemnly promise that the paper presented comes from my independent research work under my supervisor's supervision. All statistics and images are real and reliable.

2. Except for the annotated reference, the paper contents no other published work or achievement by person or group. All people making important contributions to the study of the paper have been indicated clearly in the paper.

3. I promise that I did not plagiarize other people's research achievement or forge related data in the process of designing topic and research content.

4. If there is violation of any intellectual property right, I will take legal responsibility myself.

Signature:

Date:

ABIscope —Differential Analysis of Linux ABI Compatibility in Rust-based Kernels

杨若谷

(Computer Science and Engineering Thesis Advisor: 张殷乾)

Abstract: Linux kernel is the core component of the Linux operating system, responsible for providing essential services and managing hardware resources. With the emergence of Rust as a systems programming language, there has been growing interest in using Rust for kernel development due to its memory safety features. However, ensuring ABI (Application Binary Interface) Compatibility between Rust-based kernels and existing Linux kernel modules is crucial for the successful adoption of Rust in kernel development. Existing tools like Syzkaller target at finding bugs in the Linux kernel, but they do not specifically address the challenges of ABI Compatibility in Rust-based kernels. In this thesis, we present ABIscope, a novel tool designed to generate user-space test cases for differential analysis of Linux ABI Compatibility in Rust-based kernels. ABIscope makes uses of LLMs to generate test cases. We evaluate ABIscope on Rust-based kernels and demonstrate its effectiveness in identifying ABI Compatibility issues, providing valuable insights for developers and researchers in the field of kernel development.

[Key Words]: Linux Kernel, Rust, ABI Compatibility, LLMs, Syzkaller

[摘要]：

Linux 内核是操作系统的核心组件，负责提供系统基础服务并管理硬件资源。近年来，Rust 语言由于其内存安全和线程安全的特性，在内核开发中的应用越来越受关注。然而，在现有 Linux 生态下，采用 Rust 重写或开发的内核模块必须保证与传统 C 语言内核之间的 ABI（应用程序二进制接口）兼容性。由于内核接口涉及复杂的系统调用和底层数据结构，即便没有发生崩溃，也可能产生细微的行为差异。现有的内核模糊测试工具（例如 Syzkaller）主要面向崩溃挖掘和漏洞检测，缺乏对跨内核 ABI 兼容性的直接支持。

针对上述问题，本文提出了 ABIscope——一种基于大语言模型（LLM）驱动的用户态测试用例生成与差分分析框架。我们提出了一种包含种子初始化、大模型结构与参数变异、差分执行验证以及覆盖率引导选择的闭环工作流。具体而言，本方法不依赖于内核内嵌插桩，而是设计了系统调用参数覆盖率（SAC）机制，从变异代码的静态特征和运行时的副作用等多维度进行反馈引导。

通过对比测试标准 C 语言内核与基于 Rust 构建的新型内核，ABIscope 能够自动化生成涵盖大量边界条件（Edge Cases）的系统调用序列，并结构化地分析两者在返回值、错误码及文件系统副作用上的差异。实验结果表明，ABIscope 能够有效定位并归因 ABI 不兼容问题，为未来基于 Rust 的系统内核开发和生态平滑过渡提供了有力工具和方法支持。

[关键词]： Linux 内核；Rust；ABI 兼容性；大语言模型；Syzkaller

目录

1. Introduction **1**

1.1 todo 3

2. Background and Related Work **3**

2.1 Linux kernel and Rust-based kernels 3

2.2 ABI Compatibility 4

2.3 Fuzzing and Testing Techniques 5

3. Methodology **6**

3.1 Overview 6

3.2 Seed Initialization 6

3.3 LLM-based mutation 7

3.4 Execution and Validation 8

3.5 Coverage-Guided Selection 9

3.6 Threats to Validity 10

4. Experimental Results **11**

4.1 Overall Compatibility Statistics 11

4.2 Major Incompatibility Categories 12

4.2.1 Missing Syscall Support (`openat2`) 12

4.2.2 `fcntl` Errno Precedence Mismatch (`EINVAL` vs `EBADF`) 12

4.2.3 `O_PATH` Semantics and Operation Filtering 12

4.2.4 Validation Deficits (`unexpected_success`) 12

4.3 SAC Coverage Analysis 13

5. Summary and Future Work **13**

References **14**

1. Introduction

Linux operating system has been widely adopted in various domains from personal computers to servers and embedded systems[1]. The Linux kernel is the core component of the Linux operating system. It is essential for providing different kinds of crucial services like process management,file system and so on. Linux kernel is written mostly in C, which is a powerful but unsafe programming language because of its lack of memory safety features. These days, a memory safety issue in kernels can finally result in a security vulnerability which be attacked by the malicious users and result in a severe consequence[2]. Rust is a modern programming language that provides memory safety guarantees without sacrificing performance. It has become more and more popular in the systems programming community. Researchers begin to use Rust to write kernel modules and even entire kernels like KeepOS, Asterinas, Kerla and so on[3].

However, many existing Linux kernel modules like device drivers and file systems are written in C. They can not directly work with Rust-based kernels. Based on this, ideas of using Rust to write Linux kernel modules is proposed[4]. However, ensuring ABI(Application Binary Interface) Compatibility between Rust-based kernels and existing Linux kernel modules is crucial for the successful adoption of Rust in kernel development. ABI Compatibility means that the binary interface between the kernel and user-space applications or kernel modules remains consistent, allowing them to interact seamlessly without requiring modifications[5]. Ensuring ABI Compatibility is essential for maintaining the stability and functionality of existing software while enabling the benefits of Rust's memory safety features in kernel development.

Although kernel developers have made efforts to ensure ABI Compatibility when building Rust-based kernels, there are still challenges and potential issues they can arise. For most rust-based kernels, developers just write in the documentation that the kernel they develop is ABI compatible with the Linux kernel, but they do not provide a systematic way to verify the ABI Compatibility[6]. Existing tools like Syzkaller[7] target at finding bugs in the Linux kernel, but these tools cannot directly use in the Rust-based kernels because of the differences in the kernel

implementation and the lack of support for Rust-based kernels. What's more, these tools target at finding bugs in the kernel, but finding bugs is not the same as finding ABI Compatibility issues. ABI Compatibility issues can be more subtle and may not be regarded as bugs in the kernel. In this case, there is a need for a systematic way to verify the ABI Compatibility of Rust-based kernels with existing Linux kernel modules.

Recently, large language models (LLMs) have shown great potential in various domains, including software development, code generation, and testin[8,9]. Many LLM-based test generation methods have been proposed, demonstrating strong capabilities in generating high-quality inputs and exploring complex program behaviors. However, directly applying these techniques to kernel-level ABI compatibility testing remains challenging due to the complexity of kernel interfaces and the lack of well-defined specifications[10].

In particular, the Linux ABI is highly complex, involving numerous system calls, complicated data structures, and subtle interactions between user space and kernel space. Exhaustively verifying ABI compatibility across all possible interfaces is therefore infeasible in practice. To address this challenge, we propose to focus on a critical and well-defined subset of the ABI: system calls. System calls serve as the primary interface between user applications and the kernel, and thus provide an effective abstraction for testing ABI compatibility.

Based on this insight, we design and implement ABIscope, a seed generation tool tailored for ABI compatibility testing of Rust-based kernels. ABIscope leverages LLMs to generate system call sequences (seeds) and incorporates an intermediate parameter propagation mechanism, where arguments from previous system calls are reused and transformed for subsequent calls. This design enables ABIscope to construct semantically meaningful and stateful test cases, rather than isolated and random inputs.

Furthermore, ABIscope improves system call argument coverage by systematically exploring diverse parameter combinations and dependencies. By increasing the diversity and validity of system call inputs, ABIscope indirectly enhances kernel code coverage, as measured by KCOV. This approach allows us to effectively exercise deeper execution paths in the kernel without requiring intrusive instrumentation or kernel-specific adaptations.

We evaluate ABIscope on the file system subsystem of multiple Rust-based kernels, including Asterinas and KeepOS and compare their behaviors against the Linux kernel. Our results reveal several previously undocumented ABI incompatibility issues, demonstrating that even kernels claiming ABI compatibility may exhibit inconsistencies in system call semantics and behaviors. These findings highlight the necessity of systematic ABI verification and showcase the effectiveness of ABIscope in uncovering such issues.

1.1 todo

at least three pages

2. Background and Related Work

In this section, we will review the background and related work in the areas of Linux kernel development, Rust-based kernels, ABI compatibility, fuzzing and testing techniques and also the application of large language models in the software testing field.

2.1 Linux kernel and Rust-based kernels

Linux kernel is developed based on Unix-like kernel and it is written mostly in C. The key design of the Linux kernel is monolithic kernel[11], which means that all core os services runs in a single kernel address space and interact via direct function calls. In this design, there will be no IPC, no message passing, no internal context switches and so on. IPC and context switch in microkernel design will cost microseconds level latency, but in monolithic kernel design, the latency of the function calls can be reduced to nanoseconds level. In this case, linux kernel can provide better performance than the microkernel design and this design helps the Linux kernel easy to integrate with new hardware and software. However, the monolithic kernel design also has some drawbacks. because all components share the kernel space, a bug in one component can potentially crash the entire system. Due to the complex interactions between different components, it can be difficult to isolate and debug issues. The c programming language itself is a powerful but unsafe programming language because of its lack of memory safety features. This increases the risk of security vulnerabilities and system crashes[12]. Rust, a modern systems programming language, has been gaining popularity in recent years due to

its memeory safety, concurrency safety and high performance without relying on a garbage collector[13]. It achieves this through a strong static type system and its ownership and borrowing model. Rust can prevent common memory bugs at compile time including buffer overflows, use-after-free and so on. It eliminates a large class of vulnerabilities common in C/C++[14]. Because of these advantages, there has been growing interest in using Rust for kernel development. The Linux kernel community has started to accept Rust code and there are already some Rust-based kernel like Asterinas[3], rCore[15] and Kerla[16]. However, despite these benefits, an important open question remains: whether Rust-based kernels behave consistently with the Linux kernel at the Application Binary Interface (ABI) level. Such consistency is essential for ensuring compatibility and correctness across different kernel implementations.

2.2 ABI Compatibility

ABI (Application Binary Interface) defines the low-level interface between an application and the operating system including system call conventions, data types and observable behaviors such as return values, side effects and error codes. ABI compatibility means that a program compiled staticly for a given ABI can run correctly on another kernel implementing the same ABI without modification. Also, they are expected to produce consistent observable behaviors like the same return values or the same return error codes when executing the same system calls with the same arguments[5]. In the context of kernel development, many applications and kernel modules are built on top of the Linux kernel ABI. Many programs rely on specific behaviors of the Linux kernel, some of the programs may even rely on specific return values or error codes. Many library developers use `switch` or other control flow statements to handle different return values and error codes. That is to say, even if the Rust-based kernels can successfully execute the same system calls, they may still produce different return values or error codes which may result in unexpected behaviors of the applications if they do not ensure ABI Compatibility with the Linux kernel. In this case, researchers find it crucial to evaluate the ABI Compatibility of Rust-based kernels with the Linux kernel. However, evaluating ABI Compatibility is a non-trivial task. Many kernels which claims that they are Linux-compatible seldom give a detailed

analysis of their ABI compatibility with the Linux kernel. A static analysis tool called System Call Matching Language(SCML) is proposed to analyze the syscall interface of a kernel and compare it with the Linux kernel. However, it only focuses on the flags and bits that implemented in the kernel and it does not evaluate the return values, error codes and side effects of the system calls. To the best of our knowledge, there is no existing framework or benchmark that can systematically evaluate the ABI Compatibility of Rust-based kernels with the Linux kernel.

2.3 Fuzzing and Testing Techniques

Fuzzing is a widely used technique for finding bugs and vulnerabilities in software. It involves generating random or semi-random inputs and feed them to a specific target program to trigger unexpected behaviors or crashes[17]. These kinds of programs are called fuzz drivers or fuzz targets. Fuzz drivers can be generated manually or automatically. They ensures a sequence of API calls with specific arguments to trigger specific code paths in the target program. Because developing fuzz drivers manually can be time-consuming, there are many existing researches on automatic fuzz driver generation[18,19]. Other researches focus on the input generating process and they use machine learning techniques to generate more effective inputs[20]. However, most of the existing fuzzing techniques are designed for testing user-space applications and they are not directly applicable to kernel testing. Kernel fuzzing is more challenge than user-space fuzzing because of the complexity of the kernel and the difficulty of reproducing bugs. There are some existing kernel fuzzing tools like Syzkaller[7],Trinity[21] and so on. However, these tools are designed mainly for finding bugs and vulnerabilities in the Linux kernel and it may take great effort to transpart them to do kernel fuzzing on other kernels. Also, fuzzing techniques are not designed for evaluating ABI Compatibility because they do not focus on the return values, error codes and side effects of the system calls. They find bugs by triggering crashes or hangs, but they do not evaluate the correctness of the behaviors of the system calls.

3. Methodology

3.1 Overview

We present ABIscope, a novel tool designed to generate user-space test cases for differential analysis of Linux ABI Compatibility in Rust-based kernels target at syscalls. The Overall pipeline follows an iterative seed generation and evaluation process, leveraging LLMs with feedback-driven exploration. At a high level, ABIscope consists of four key stages: (1) Seed initialization (Section 3.2), (2) LLM-based mutation (Section 3.3), (3) execution and validation (Section 3.4), and (4) coverage-guided selection (Section 3.5). These four key components form a closed feedback loop. The overall pipeline is illustrated in Figure 1. We describe each stage in detail in the following sections.

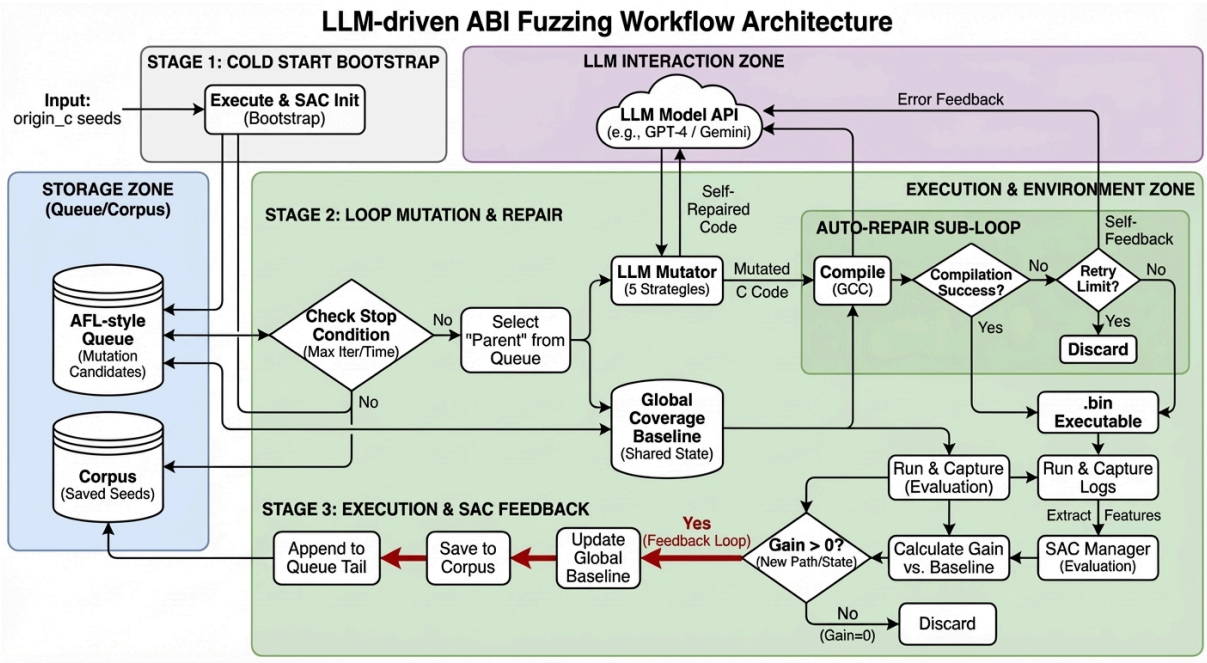

Figure 1 Overall pipeline of ABIscope.

3.2 Seed Initialization

Linux ABI Compatibility is complicated by the fact that the Linux kernel provides a large number of system calls and different structures, each of them have their own ABI. It is impractical to generate test cases for all of them. To simplify the problem, we focus on a subset of system calls that are commonly used and have well-defined ABIs. We select a set of system

calls of file system, involves `open`, `read`, `write`, `close`, `mmap`, `munmap`, `ioctl` and so on. The detailed definition of these syscalls can be found in the appendix.

Syzkaller is a popular fuzzing tool for finding bugs in the Linux kernel developed by Google. It generates random inputs for system calls and monitors the kernel's behavior to identify potential vulnerabilities. But we cannot directly use Syzkaller to generate test cases for ABI Compatibility testing because incompatibility issues may not necessarily cause crashes or other easily detectable symptoms[22]. That is to say, Syzkaller does not monitor the output of the system calls, which is crucial for identifying ABI Compatibility issues. However, Syzkaller can be used to generate initial seed test cases for our tool. We use syzkaller to generate a large number of test cases for the selected system calls and then filter out the valid ones based on their output. These valid test cases should be able to execute successfully on the Linux kernel and produce expected results. After the filtering process, we use a pre-written script to print the syscalls return value and the output parameters in a structured format. These output will be used to evaluate the compatibility of the test cases on Rust-based kernels.

3.3 LLM-based mutation

ABIscope adopts a mutation-based approach to generate new test cases from the initial seeds. We leverage LLMs to perform mutations on the valid test cases generated in the previous stage. The LLM taks the valid test cases as input and generates new test cases by applying various mutation strategies.

- Structure mutation: LLM can insert new syscalls to the test cases which can trigger new code paths in the kernel. For example, LLM can insert `mmap` syscall before `read` syscall to trigger the code path of memory mapping in the kernel. Also, LLM can remove some syscalls from the test cases to reach the edge cases. For example, LLM can remove `open` syscall from the test case to trigger the code path of handling invalid file descriptors to see the ability of the error-handling code in the rust-based kernel.

- Order mutation: LLM can change the order of the syscalls in the tests cases to trigger different code paths in the kernel. For example, LLM can change the order of `rmdir` and `rename` syscalls to trigger the code path of handling directory operations in the kernel.
- Fault injection: LLM can inject faults into the test cases to trigger specific error-handling code paths in the kernel. We can find incompatibility issues through the different errno values returned by the syscalls.
- Logic mutation: LLM can construct stateful misuse patterns like double free, use-after-free and so on. The idea is the same as the logic mutation in AFL++ to see the compatibility of error-handling.
- Parameter mutation: LLM perturbs syscall arguments at both value and structure levels to expose ABI-sensitive boundary behavior. Concretely, it rewrites scalar parameters (e.g., file descriptors, offsets, lengths, and flags) toward the cases that never appear in the seeds before. For instance, it may generate negative offsets, oversized lengths, or uncommon flag combinations. LLM can effectively explore the argument space by learning from the existing seeds.

3.4 Execution and Validation

After generating new test cases through LLM-based mutation, ABIscope executes these test cases on Linux first. If the test case can be executed successfully on Linux, we will execute the same test case on Rust-based kernels. If the generated programs cannot be executed successfully on Linux, we will involve another LLM to fix the test cases. If the repair process fails after limit attempts, we will discard the test cases. The execution results of the test cases on both Linux and Rust-based kernels will be collected and compared to evaluate the compatibility of the test cases. We will compare the return values, output parameters, and error codes of the syscalls to identify any discrepancies that may indicate ABI incompatibility issues. If we find any discrepancies, we will further analyze the test cases to understand the root cause of the incompatibility and write reports to summarize the issues we found.

3.5 Coverage-Guided Selection

To quantify the effectiveness of LLM-generated mutations, ABIscope adopts SAC (System-call Argument Coverage) as its primary feedback signal. In contrast to conventional control-flow coverage (e.g., line, basic-block, or branch coverage collected through kernel instrumentation), SAC evaluates whether a mutated seed explores previously unseen argument semantics and externally observable execution outcomes. This formulation is particularly suitable for ABI compatibility analysis, where behavioral divergence may arise even when no crash or obvious control-flow anomaly is observed.

In our implementation, SAC integrates both static and dynamic evidence. The static dimension is extracted directly from generated C programs and includes: (i) flag-level behaviors, where individual macros (such as `O_RDWR`, `O_CREAT`, and `SEEK_END`) and their co-occurrence relations are encoded; (ii) path-pattern abstractions, where path-like literals are mapped to semantic categories defined by preconfigured pattern rules; (iii) numeric and alignment-sensitive arguments, including boundary values (e.g., `-1`, `0`), page-aligned constants, and oversized unsigned probes; and (iv) permission-mask semantics, where mode-related expressions are matched against predefined privilege-related masks to detect bits such as SUID/SGID.

The dynamic dimension is obtained from runtime logs after compilation and execution of each seed. Through syscall-result probes (e.g., `LOG_SYSCALL_RES`), the framework records ordered syscall occurrences, per-call success/failure status, concrete errno outcomes, and global errno visibility. In addition, side effects on filesystem objects are characterized through `stat`-derived attributes, including file size regimes, link-count transitions, and effective mode changes. These dynamic observations provide direct evidence of kernel-visible state transitions and therefore complement static argument analysis.

All extracted signals are normalized into a unified feature set. For a new seed, ABIscope computes coverage gain as the cardinality of the set difference between the seed feature set and the accumulated global coverage set (equivalently, `gain = len(new_features -`

`global_coverage)`). Seeds with positive gain are admitted into the corpus and prioritized for subsequent mutation rounds, while seeds with zero gain are typically discarded. Consequently, SAC serves as a non-instrumentation, semantics-oriented guidance mechanism that continuously steers the LLM toward high-value edge cases for ABI incompatibility discovery.

Also, the mutation strategies are not applied randomly. Instead, we use a reinforcement learning-based approach to select the mutation strategy for each seed. The reward signal is based on the coverage gain of the mutated seed. The mutation strategy that leads to higher coverage gain will be selected more frequently in the future. This approach allows ABIscope to adaptively focus on the most promising mutation strategies and improve the efficiency of selection process. The overall pipeline will be repeated until it reaches the maximum number of iterations or the time budget is exhausted.

3.6 Threats to Validity

Despite the feedback-driven design of ABIscope, several validity threats still remain.

First, there are construct-validity threats in the coverage signal. SAC is designed to capture argument semantics and observable side effects, but it is still a proxy metric rather than a direct measure of incompatibility severity. A seed may produce positive SAC gain without exposing a meaningful ABI defect, while some true incompatibilities may exhibit limited gain if their behavioral signatures overlap with already observed patterns. To reduce this risk, SAC is evaluated jointly with differential execution outcomes and error-code distributions, rather than used as a standalone criterion.

Second, there are internal-validity threats in mutation and execution. LLM-based mutation is inherently stochastic and can be sensitive to prompt context, which may affect reproducibility across runs. In addition, generated tests can contain invalid or weakly constrained arguments; such cases may amplify input noise and obscure real kernel-level divergence. We therefore retain only compilable/runnable candidates and interpret failures together with runtime traces, so that parameter invalidity can be separated from compatibility-relevant behavioral differences.

Third, differential execution itself introduces confounding factors. Linux and Rust-based kernels may differ in scheduling behavior, timing, default configuration, and tolerance to undefined or implementation-dependent behavior. Some output discrepancies can therefore reflect environmental or semantic ambiguity rather than strict ABI incompatibility. To mitigate this issue, ABIscope compares aligned test inputs under controlled conditions and emphasizes stable, repeatable divergences.

Finally, there are external-validity limitations. The current setup focuses on a selected file system-related syscall subset and may not fully generalize to other subsystems (e.g., networking, IPC, device drivers) or production workloads. Consequently, conclusions should be interpreted as evidence for the studied syscall domain, with broader generalization requiring expanded syscall coverage and larger-scale cross-kernel evaluation.

4. Experimental Results

This chapter focuses on the differential results of file-system-related system calls (e.g., `openat`, `read`, `write`, `fcntl`, `renameat`) when comparing native Linux with two Rust-based OS kernels: KeepOS and Asterinas. We ran an identical corpus of 500 generated C programs, built natively on Linux, and executed them across all three operating environments with sandboxing.

4.1 Overall Compatibility Statistics

Both KeepOS and Asterinas demonstrate highly analogous behavior as shown by the number of incompatible test cases out of the 500 executions.

Metric	KeepOS	Asterinas
Total Analyzed Cases	500	500
Incompatible Cases	81	80

The distribution of incompatibility types is strongly correlated between the two systems. The most frequent issues are the divergence of return error codes (`errno_diff`) and behavioral differences (`behavior_diff`).

Incompatibility Type	KeepOS Cases	Asterinas Cases
`errno_diff`	79	76
`behavior_diff`	33	32

ret_diff	26	25
unexpected_failure	21	20
unexpected_success	21	15
missing_syscall	11	10

4.2 Major Incompatibility Categories

4.2.1 Missing Syscall Support (openat2)

Across 10 test cases, both KeepOS and Asterinas return ENOSYS (38) for the openat2 syscall (syscall 437). Linux, depending on the test arguments, typically returns EINVAL (22) if the struct size or flag combinations are malformed. This highlights a gap indicating neither system currently supports the Linux 5.6+ openat2 ABI endpoint.

4.2.2 fcntl Errno Precedence Mismatch (EINVAL vs EBADF)

A highly frequent defect (affecting 31 cases in KeepOS and 22 in Asterinas) revolves around fcntl argument validation priority. When an operation is performed on an invalid file descriptor (e.g., -1) and an unsupported command is passed, the Linux VFS checks the file descriptor logic first, returning EBADF (9). Conversely, KeepOS and Asterinas evaluate the fcntl command first and reject it as an unsupported operation, returning EINVAL (22).

4.2.3 O_PATH Semantics and Operation Filtering

Linux enforces strict semantics for file descriptors instantiated with the O_PATH flag: most operations (including many fcntl commands) are invalid on them and immediately yield EBADF. The test results reveal 11 failures in KeepOS and 9 in Asterinas where operations on O_PATH descriptors behaved differently (yielding EINVAL instead of the required EBADF or proceeding successfully). The Rust-based kernels currently fail to maintain identical filtering boundaries.

4.2.4 Validation Deficits (unexpected_success)

There is looser enforcement of access mode flags in the new kernels. While Linux rigorously verifies the flags parameter (e.g., bits mapping to O_ACCMODE forming invalid states on specific requests), Asterinas and KeepOS sometimes accept and quietly overlook invalid flags. Notably, Asterinas handles invalid memory pointers and boundary checks mildly tighter

than KeepOS (yielding 15 unexpected successes vs KeepOS's 21), but both diverge from Linux's rigorous argument validation.

4.3 SAC Coverage Analysis

5. Summary and Future Work

This paper constructs an ABI differential analysis workflow for Rust-based kernels, forming a closed loop of "sample generation — dual-platform execution — log alignment — LLM attribution". The current research primarily focuses on a single kernel subsystem (e.g., the file system and VFS-related system calls). Future work includes:

- Expanding the testing and analysis framework to multiple kernel subsystems (such as the network stack, process management, and memory mapping);
- Further enlarging the test sample dataset to improve parameter coverage and the probability of triggering edge cases;
- Not only performing automated attribution for discovered incompatibilities, but also generating concrete fix suggestions directly for developers;
- Expanding the regression testing for other kernels, such as `kerla` and `rcore`.

References

[1] WANG Y J. A Summary of Research on the Linux Operating System[J/OL]. Advanced Materials Research, 2013, 705: 565-568. https://api.semanticscholar.org/CorpusID:62728732.

[2] SZEKERES L, PAYER M, WEI T, et al. SoK: Eternal War in Memory[C/OL]//Proceedings of the 2013 IEEE Symposium on Security and Privacy. USA: IEEE Computer Society, 2013: 48-62. https://doi.org/10.1109/SP.2013.13.

[3] PENG Y, TIAN H, ZHANG J, et al. ASTERINAS: a Linux ABI-compatible, rust-based framekernel OS with a small and sound TCB[C]//Proceedings of the 2025 USENIX Conference on Usenix Annual Technical Conference. Boston, MA, USA: USENIX Association, 2025.

[4] CHEN S F, WU Y S. Linux Kernel Module Development with Rust[C]//2022 IEEE Conference on Dependable and Secure Computing (DSC). 2022: 1-2.

[5] SOCHAT V, HAINES T. Binary-level Software Compatibility Tool Agreement[EB/OL]. (2023). https://arxiv.org/abs/2212.03364.

[6] BOOS K, LIYANAGE N, IJAZ R, et al. Theseus: an Experiment in Operating System Structure and State Management[C/OL]//14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20). USENIX Association, 2020: 1-19. https://www.usenix.org/conference/osdi20/presentation/boos.

[7] LAN Y, JIN D, WANG Z, et al. Thunderkaller: Profiling and Improving the Performance of Syzkaller[C/OL]//Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering. Echternach, Luxembourg: IEEE Press, 2024: 1567-1578. https://doi.org/10.1109/ASE56229.2023.00124.

[8] CHEN M, TWOREK J, JUN H, et al. Evaluating Large Language Models Trained on Code[EB/OL]. (2021). https://arxiv.org/abs/2107.03374.

[9] LU S, GUO D, REN S, et al. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation[EB/OL]. (2021). https://arxiv.org/abs/2102.04664.

[10] CHEN H, MAO Y, WANG X, et al. Linux kernel vulnerabilities: state-of-the-art defenses and open problems[C/OL]//Proceedings of the Second Asia-Pacific Workshop on Systems. Shanghai, China: Association for Computing Machinery, 2011. https://doi.org/10.1145/2103799.2103805.

[11] LOVE R. Linux kernel development[M]. Pearson Education, 2010.

[12] RESHETOVA E, LILJESTRAND H, PAVERD A, et al. Toward Linux kernel memory safety[J/OL]. Software: Practice and Experience, 2018, 48(12). http://dx.doi.org/10.1002/spe.2638.

[13] BUGDEN W, ALAHMAR A. Rust: The Programming Language for Safety and Performance[EB/OL]. (2022). https://arxiv.org/abs/2206.05503.

[14] KAN S, CHEN Z, SANAN D, et al. An Executable Operational Semantics for Rust with the Formalization of Ownership and Borrowing[EB/OL]. (2020). https://arxiv.org/abs/1804.07608.

[15] RCORE-OS. rCore[EB/OL]. (2026). https://github.com/rcore-os/rCore.

[16] NUTA. Kerla[EB/OL]. (2026). https://github.com/nuta/kerla.

[17] BOEHME M, CADAR C, ROYCHOUDHURY A. Fuzzing: Challenges and Reflections[J]. IEEE Software, 2021, 38(3): 79-86.

[18] LYU Y, XIE Y, CHEN P, et al. Prompt Fuzzing for Fuzz Driver Generation[C/OL]//Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. Salt Lake City, UT, USA: Association for Computing Machinery, 2024: 3793-3807. https://doi.org/10.1145/3658644.3670396.

[19] LIU Y, DENG J, JIA X, et al. PromeFuzz: A Knowledge-Driven Approach to Fuzzing Harness Generation with Large Language Models[C/OL]//Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security. Taipei, Taiwan: Association for Computing Machinery, 2025: 1559-1573. https://doi.org/10.1145/3719027.3765222.

[20] GODEFROID P, PELEG H, SINGH R. Learn&Fuzz: machine learning for input fuzzing[C]//Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering. Urbana-Champaign, IL, USA: IEEE Press, 2017: 50-59.

[21] JONES D. Trinity: A Linux System Call Fuzzer[M]. 2010.

[22] LEFEUVRE H, GAIN G, BĂDOIU V A, et al. Loupe: Driving the Development of OS Compatibility Layers[C/OL]//Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1. La Jolla, CA, USA: Association for Computing Machinery, 2024: 249-267. https://doi.org/10.1145/3617232.3624861.